

Memory Management

TLBs

CS232 Operating Systems Fall 2019

Taj Muhammad Khan

Paging

- Solves the problem of external fragmentation
- Is very slow
 - Have to access page table in RAM
 - For every memory access

Speeding up paging

- Accomplished via Hardware Support
- TLB works as a cache for frequently performed V.A-to-P.A translations
- Part of MMU
- Upon accessing a Virtual Address, before accessing a page table:
 - The hardware first tries to find a VPN in the TLB
 - If successful, it uses the corresponding PFN (thus saving the access to page table)
 - If unsuccessful, it goes and accesses the page table in RAM to get the PFN
 - Updates the TLB

TLB Basic Algorithm

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset    = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                                // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19     RetryInstruction()
```

Figure 19.1: TLB Control Flow Algorithm

TLB – example

- 8 bit address space
- 16 byte pages
- Array of 10 ints(4 bytes each)

```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

	Offset				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 02					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

TLB – example ... contd.

Initially assume that the TLB is empty.

- Access a[0] : VA=100
- Access a[1] : VA=104
- Access a[2] : VA=108
- Access a[3] : VA=112
- Access a[4] : VA=116
-
- TLB hit rate = ??%
- What if we change page size?

	Offset				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 02					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

Who handles a TLB miss?

- CISC
 - Usually hardware
 - Reads PFN, updates TLB, and retries the insn
- RISC
 - Software (OS)
 - On a TLB miss, the hardware raises exception
 - The OS exception handler reads PFN from page table in memory
 - Updates the TLB
 - `return_from_trap` (exception)
 - This `return_from_trap` is different from the `return_from_trap` of a system call
 - What if the insn that accesses a page table for PFN results in TLB miss?

TLB contents

- Typically 32, 64, or 128 entries
- Fully associative
- A search has to be performed (done in parallel)
- Other bits:
 - Valid bit
 - Protection bits
 - Dirty bit
 - Address Space ID

VPN | PFN | other bits

TLB issue – context switches

- P1 VPN 10 mapped to PFN 100
- P2 VPN 10 mapped to PFN 170

VPN	PFN	valid	prot
10	100	1	rwX
—	—	0	—
10	170	1	rwX
—	—	0	—

- Solution:
 - 1. Flush the TLB (set all valid bits to 0)
 - Costly. Misses each time context is switched
 - 2. ASID
 - Needs hardware support. ASID of current process has to be updated on context switch

VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
—	—	0	—	—
10	170	1	rwX	2
—	—	0	—	—

TLB issue: replacement policy

- Associative cache
- When all entries are full and a new one arrives
- Which one to replace?
 - LRU
 - Corner case: loop over $n+1$ pages in a n -entry page table
 - Random
 - Simple to implement

TLB entry: example

- MIPS R4000
- 32 bit address space
- 4KB pages
- Global bit (G) for globally shared pages. ASID ignored.

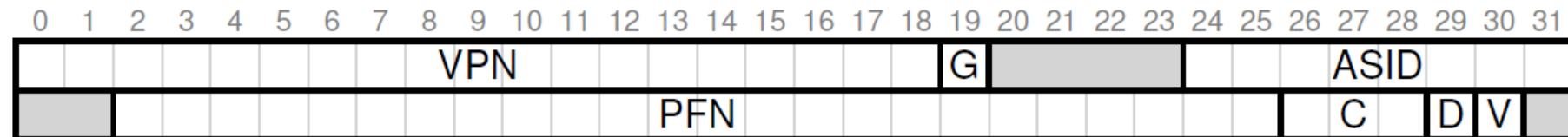


Figure 19.4: A MIPS TLB Entry